- 딥러닝
- = 손실 함수를 최소화 하는 거임

- **Gradient Descent**가 쓰임
- 최적인 세타를 찾아야 함.
- 기울기의 반대 방향으로 이동하며 → 손실 함수의 최소값을 찾음.
- $\theta \leftarrow \theta - \eta \nabla L(\theta)$
- $\nabla L(\theta)$ → 손실 함수가 가장 빠르게 증가하는 방향
- $-\nabla L(\theta)$ → 손실 함수가 가장 빠르게 감소하는 방향
- η → learning rate (한 번에 이동하는 크기 조절)
- 전역 최소값에 도달하는 것은 보장되지 않음.

$$\theta^{k+1} \leftarrow \theta^k - \eta \nabla_\theta J(\theta^k)$$

- → one iteration.

Stochastic Gradient Descent(SGD)

- 데이터가 너무 많아서, 학습하기 힘들 때가 많음
- 전체 데이터 대신 일부 데이터를 무작위로 뽑아서 gradient를 계산함.
- 엄밀한 SGD → 샘플 1개로 gradient 계산
- Mini-batch GD → 여러 샘플로 이루어진 작은 batch로 gradient 계산
- 전체 데이터 N개에 대한 평균 손실이 $L(\theta) = (1/N) \Sigma_{i=1}^{N} \ell_i(\theta)$ 이면,
- 전체 gradient: $g = (1/N) \Sigma_{i=1}^{N} \nabla \ell_i(\theta)$
- Mini-batch gradient: $g_B = (1/|B|) \Sigma_{i \in B} \nabla \ell_i(\theta)$
- 무작위로 균등 추출하면 → $E[g_B] = g$
- 즉, mini-batch gradient는 전체 gradient의 불편추정량(unbiased estimator)임.
- 매번 똑같거나 비슷하다고 가정하는 게 아니라, 기댓값이 같다는 뜻임.
- 일부 데이터만 계산하므로 한 번의 업데이트 비용이 작음.
- 하지만 sampling에 따른 분산이 있어서 → 업데이트 방향이 흔들리고 학습이 불안정할 수 있음.

If batchsize=1 then θ is updated after each example.

If batchsize=N (full set) then this is standard gradient descent.

- 이렇게 하면 빠름. + 로컬 minimum에서 벗어날 수 있음
  좀 불안정함.

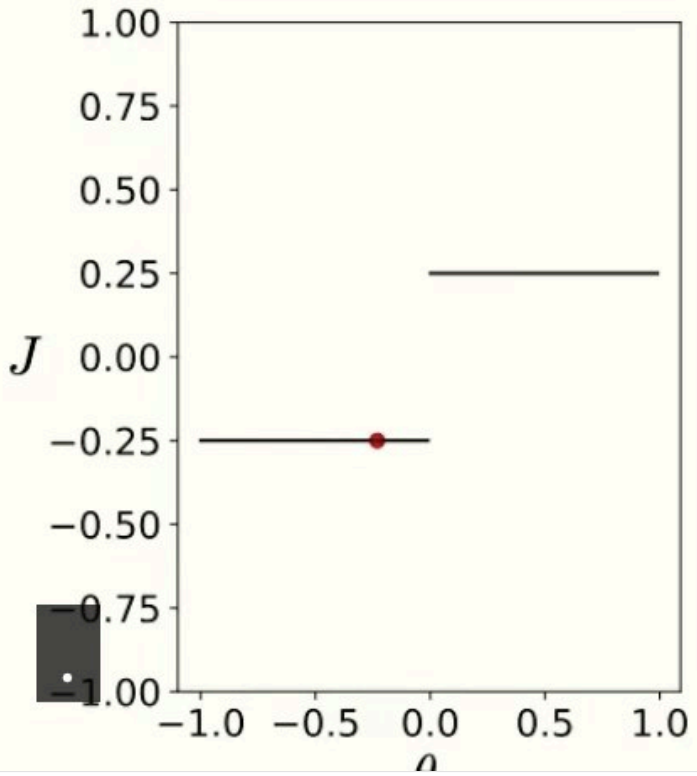
Momentum

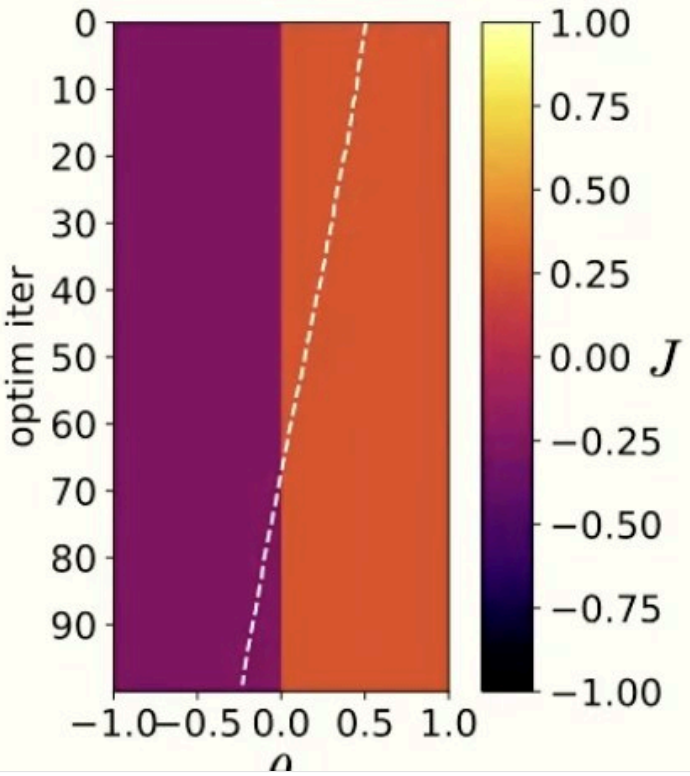
- GD에서 업데이트 방향으로 **관성**을 가지고 업데이트 하게 됨

$$\theta^{t+1} \leftarrow \theta^t - \eta \nabla f(\theta^t) - \alpha m^t$$

- 캡처 3의 Momentum 업데이트 식은 다음과 같음.
- $\theta^{t+1} \leftarrow \theta^t - \eta \nabla f(\theta^t) - \alpha \cdot m^t$
- 여기서 η는 learning rate, α는 이전 gradient의 영향을 얼마나 유지할지 결정하는 momentum 계수임.
- m^t는 이전 단계들에서 계산한 gradient에 learning rate를 곱한 값을 누적한 양임. 과거 gradient의 영향은 α에 의해 점차 감소함.
- 캡처의 부호에 맞춰 다음과 같이 정의함.
- $m^0 = 0$
- $m^{t+1} = \eta \nabla f(\theta^t) + \alpha \cdot m^t$
- 따라서 $t \geq 1$에서 다음 관계가 성립함.
- $m^t = \theta^{t-1} - \theta^t$
- 즉, m^t는 이전 파라미터 이동량 $\theta^t - \theta^{t-1}$의 음수임.
- 과거 gradient와 현재 gradient의 방향이 비슷하면, $-\alpha \cdot m^t$ 항이 현재의 gradient descent 방향에 더해져 관성이 생김.
- 주의: 이 캡처의 m^t는 파라미터의 이동 방향을 그대로 누적하는 velocity와 부호가 반대임. 따라서 일반적인 momentum의 velocity 정의를 그대로 대입하면 캡처 식과 부호가 맞지 않음.
- 너무 작은 모멘텀 → 최소값에 적당히 천천히 감
  적당한 모멘텀 → 최소값에 더 빨리 도달 할 수 있음
  너무 높은 모멘텀 → 최소값을 지나가버림

Evolution Strategies





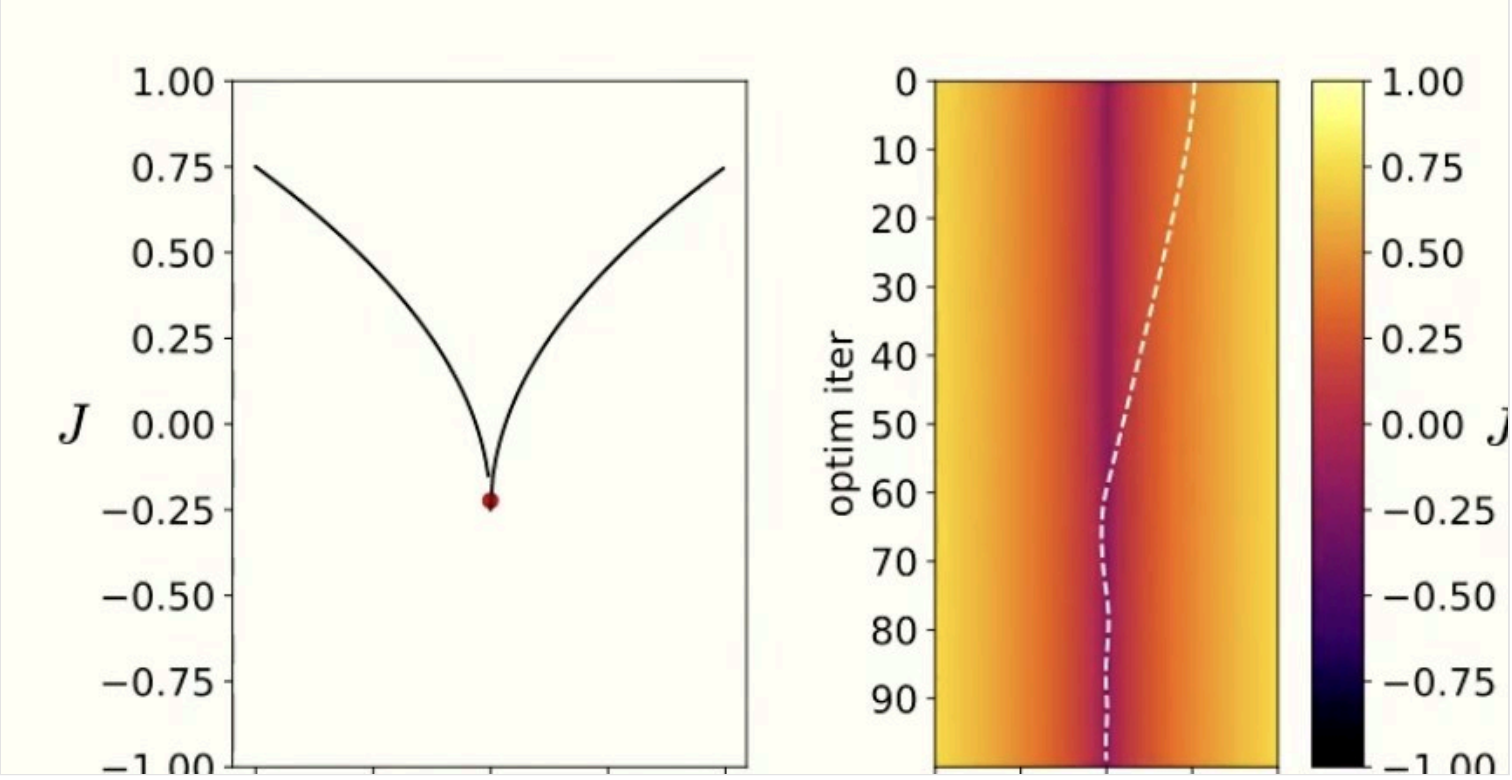
$$\epsilon_i \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$
$$s_i = J(\theta + \sigma\epsilon_i)$$

$$\theta^{k+1} \leftarrow \theta^k - \eta \frac{1}{\sigma M} \sum_{i=1}^{M} s_i \epsilon_i$$

- Evolution Strategies(ES)는 목적함수의 gradient를 직접 계산하지 않고, 파라미터를 여러 방향으로 조금씩 변화시켜 얻은 목적함수 값으로 업데이트 방향을 추정하는 방법임.
- 캡처 4처럼 목적함수 J가 계단형으로 불연속인 경우, 대부분의 구간에서 gradient가 0이고 불연속 지점에서는 미분이 정의되지 않음.
- 따라서 일반적인 gradient descent로는 유용한 업데이트 방향을 구하기 어려울 수 있음.
- ES에서는 현재 파라미터 θ 주변에 무작위 노이즈를 추가하여 목적함수 값을 평가함.
- 캡처 5의 식은 다음과 같음.
- $\varepsilon_i \sim N(0, I)$
- $s_i = J(\theta + \sigma\varepsilon_i)$
- $\theta^{k+1} \leftarrow \theta^k - \eta \cdot (1/(\sigma M)) \cdot \Sigma_{i=1}^{M} s_i \cdot \varepsilon_i$
- 여기서
- ε_i → 표준 정규분포에서 추출한 무작위 노이즈 벡터
- σ → 파라미터에 추가하는 노이즈의 크기
- M → 평가에 사용하는 노이즈 샘플의 개수
- s_i → 노이즈를 적용한 파라미터에서 계산한 목적함수 값
- 각 노이즈 방향 ε_i에 목적함수 값 s_i를 가중치로 곱하고, 여러 샘플의 결과를 평균 내어 gradient 방향을 추정함.
- 즉, 다음과 같은 추정량을 사용함.

- $\hat{g} = (1/(\sigma M)) \cdot \Sigma_{i=1}^{M} s_i \cdot \varepsilon_i$
- 이 추정량은 원래 목적함수 J 자체의 gradient가 아니라, Gaussian noise로 평활화한 목적함수의 gradient를 추정한 것임.
- $J_\sigma(\theta) = E_\varepsilon[J(\theta + \sigma\varepsilon)]$
- 적절한 조건에서 다음이 성립함.
- $\nabla J_\sigma(\theta) = (1/\sigma)\, E_\varepsilon[J(\theta + \sigma\varepsilon)\varepsilon]$
- 따라서 J가 불연속이더라도 평활화된 목적함수를 기준으로 업데이트 방향을 추정할 수 있음.
- 캡처 5의 업데이트는 목적함수 J를 최소화하는 경우를 기준으로 하며, 추정한 gradient의 반대 방향으로 파라미터를 이동함.
- 즉, 함수의 미분값을 직접 구하는 대신 여러 perturbation에 대한 평가 결과를 이용하는 gradient-free optimization 방법임.

Gradient clipping



- 기울기가 너무 커지면 → 기울기 자체를 제한함.
- 매개변수 변화량이 아니라 gradient가 1차적인 clipping 대상임.
- $g = \nabla L(\theta)$, $c > 0$ (clipping threshold)
- **Norm clipping**
- $\|g\| > c$이면 전체 gradient를 재조정함.
- $g_{clip} = g \cdot \min(1, c/\|g\|)$
- → gradient의 norm을 최대 c로 제한함.
- → 원래 gradient의 방향은 유지됨.
- **Value clipping**
- 각 gradient 성분을 [-c, c] 범위로 제한함.

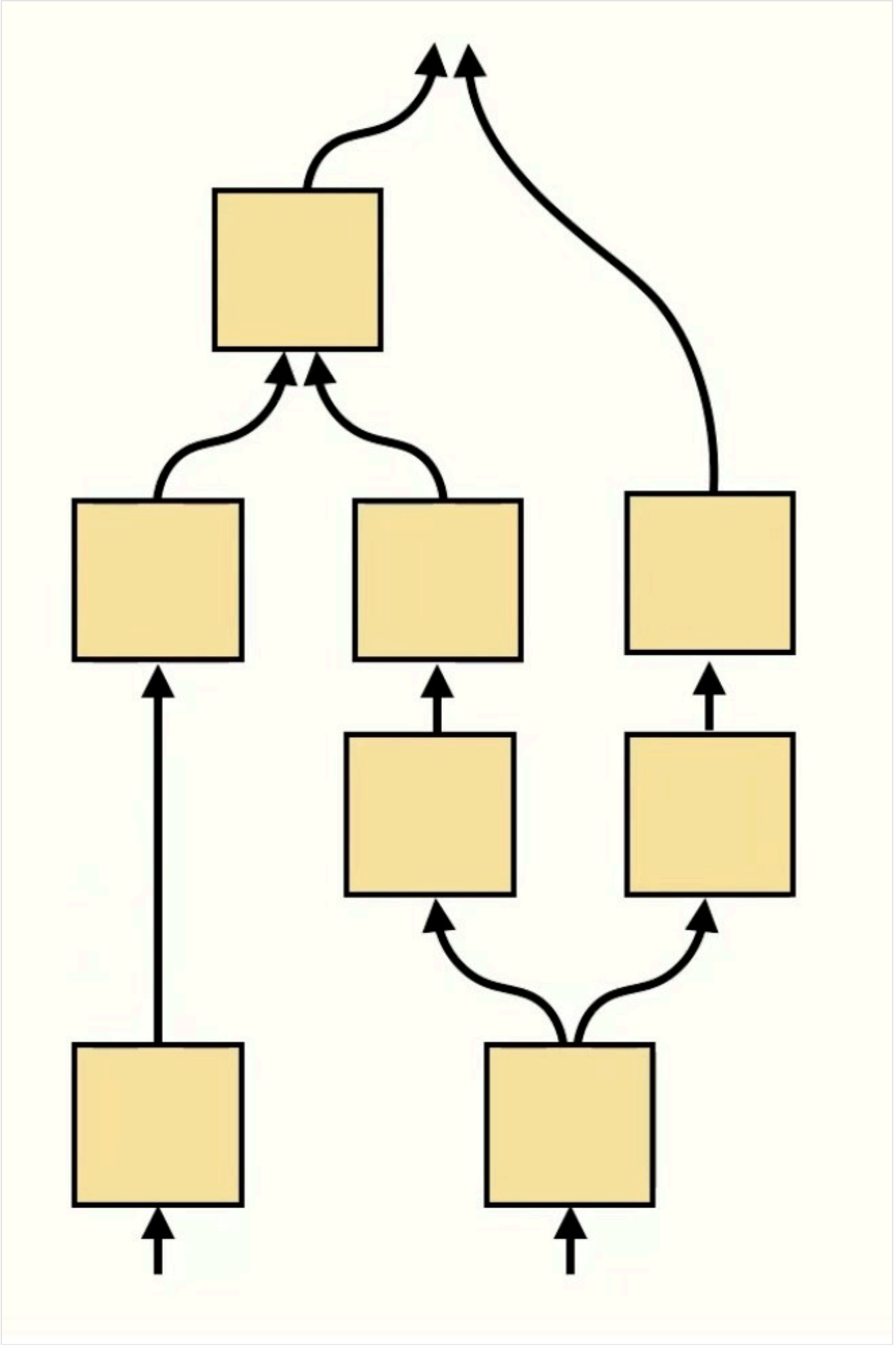
- $(g_{clip})_i = \max(-c, \min(c, g_i))$
- → 각 성분을 개별적으로 잘라냄.
- → 전체 gradient의 방향은 바뀔 수 있음.
- Clipping한 gradient로 매개변수를 업데이트함.
- $\theta \leftarrow \theta - \eta g_{clip}$
- 즉, 너무 큰 gradient 때문에 업데이트가 과도해지는 것을 방지하는 방법임.

$$\mathbf{v} = \nabla_\theta J(\theta^k)$$
$$\theta^{k+1} \leftarrow \theta^k - \eta[\mathtt{clip}(v_1, -m, m), \ldots, \mathtt{clip}(v_M, -m, m)]^\mathsf{T}$$

Loss function

- 손실 함수를 최소화하기 위해 gradient를 계산함.
- 전 구간에서 미분 가능하거나 smooth할 필요는 없음.
- 대부분의 점에서 미분 가능하면 gradient 기반 학습이 가능하고, 미분 불가능한 점에서도 부분기울기(subgradient)가 정의되면 이를 이용할 수 있음.
- EX> ReLU = max(0, z)
- ReLU는 손실 함수가 아니라 활성화 함수(activation function)임.
- z = 0에서 미분 불가능하지만, 해당 지점에서 사용할 기울기 값을 정해 학습할 수 있음.
- GELU는 smooth한 활성화 함수임.
- ReLU 대신 GELU를 쓰기도 하지만, 선택 이유가 smooth함 하나만은 아님.

Computation Graphs



- 아래가 input, 위로 갈수록 output
- Computation Graph에서 하나의 노드 출력이 여러 경로로 분기될 수 있음.
- Forward pass에서는 같은 값이 여러 연산에 입력으로 사용됨.

- Backward pass에서는 각 경로에서 전달되는 gradient의 기여를 모두 합산해야 함.
- 예를 들어 스칼라 값 z가 두 경로로 나뉘어 다음과 같이 계산된다고 하자.
- $a = h_1(z)$
- $b = h_2(z)$
- $J = J(a, b)$
- 이때 chain rule에 의해 z에 대한 gradient는 다음과 같음.
- $\partial J/\partial z = (\partial J/\partial a)(\partial a/\partial z) + (\partial J/\partial b)(\partial b/\partial z)$
- 첫 번째 항은 z → a → J 경로를 통한 기여이고, 두 번째 항은 z → b → J 경로를 통한 기여임.
- 즉, 하나의 노드에서 여러 경로로 분기되었다면 backward 과정에서는 각 경로에서 돌아온 gradient를 더한 뒤 이전 노드로 전달함.
- 경로마다 chain rule을 적용하고, 같은 노드로 모이는 gradient는 합산하는 것이 핵심임.

Backpropagation

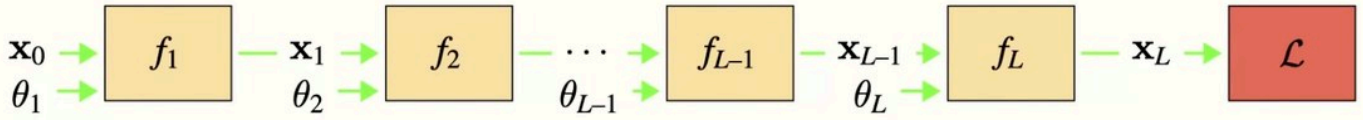
$$\frac{\partial J}{\partial \theta_1} = \frac{\partial J}{\partial \mathbf{x}_L}\frac{\partial \mathbf{x}_L}{\partial \mathbf{x}_{L-1}}\cdots\frac{\partial \mathbf{x}_3}{\partial \mathbf{x}_2}\frac{\partial \mathbf{x}_2}{\mathbf{x}_1}\frac{\partial \mathbf{x}_1}{\partial \theta_1}$$

$$\frac{\partial J}{\partial \theta_2} = \frac{\partial J}{\partial \mathbf{x}_L}\frac{\partial \mathbf{x}_L}{\partial \mathbf{x}_{L-1}}\cdots\frac{\partial \mathbf{x}_3}{\partial \mathbf{x}_2}\frac{\partial \mathbf{x}_2}{\partial \theta_2}$$

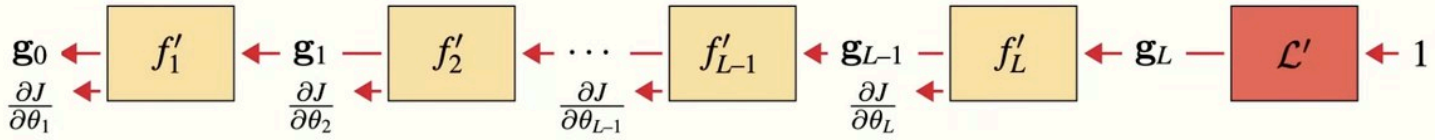
- 정정: 슬라이드의 연쇄법칙 식에서 끝에서 두 번째 인자인 $\partial x_2/x_1$은 분모의 ∂가 누락된 오기입니다. 올바른 표기는 $\partial x_2/\partial x_1$입니다.
- Computation Graphs에서 세타에 대한 미분을 계산하기 위한 방법.
  Chain rule을 이용해 backward 방향으로 미분값을 얻음

The Full Algorithm: Forward, Then Backward

Forward:



Backward:



Update:

$$\theta^{i+1} \leftarrow \theta^i - \eta \left(\frac{\partial J}{\partial \theta} \right)^{\mathsf{T}}$$

… and repeat

- forward → backward → update 반복하면 training

- Linear layer
- Forward → f = Wx + b
- W → weight matrix
- x → input vector
- b → bias vector
- f → output vector
- 입력에 W를 곱하고 bias b를 더함.
- bias까지 포함하면 수학적으로는 affine transformation임.
- b = 0이면 f = Wx가 됨.
- Backprop

$$\mathbf{g}_{\text{in}} = \mathbf{g}_{\text{out}} \cdot \frac{\partial f(\mathbf{x}_{\text{in}}, \mathbf{W})}{\partial \mathbf{x}_{\text{in}}} = \mathbf{g}_{\text{out}} \cdot \frac{\partial \mathbf{x}_{\text{out}}}{\partial \mathbf{x}_{\text{in}}}$$

$$\frac{\partial \mathbf{x}\mathbf{out}_i}{\partial \mathbf{x}\mathbf{in}_j} = \mathbf{W}_{ij} \longrightarrow \frac{\partial f(\mathbf{x}\mathbf{in}, \mathbf{W})}{\partial \mathbf{x}\mathbf{in}} = \mathbf{W}$$

- therefore

$$\mathbf{g}\mathbf{in} = \mathbf{g}\mathbf{out} \cdot \mathbf{W}$$

- ※ 표기 주의: 캡처 13의 $g_{in} = g_{out} \cdot W$는 입력과 출력의 gradient를 행벡터(row vector)로 표현한 식임.
- 반면 위에서 정의한 forward 식 $f = Wx + b$는 입력 x와 출력 f를 열벡터(column vector)로 두는 관례임.
- 이 열벡터 관례에서는 다음과 같이 계산함.
- $g_{out} = \partial J / \partial f$
- $g_{in} = \partial J / \partial x = W^T g_{out}$
- 즉, 입력 쪽 gradient를 계산할 때 weight matrix W의 전치행렬을 곱함.
- 캡처의 $g_{in} = g_{out} \cdot W$와 본문의 $g_{in} = W^T g_{out}$은 gradient를 행벡터로 쓰는지 열벡터로 쓰는지에 따른 표기 차이이며, 같은 chain rule을 나타냄.
- 여기서는 loss function 자체를 새로 구하는 것이 아님.
- Forward pass에서 계산된 손실 J를 기준으로, backward pass에서는 손실의 gradient를 출력 쪽에서 입력 쪽으로 전파함.
- Linear layer에서는 출력 f에 대한 손실의 gradient $g_{out} = \partial J / \partial f$를 전달받고, chain rule을 이용해 입력 x에 대한 gradient를 계산함.

$$\frac{\partial J}{\partial \mathbf{W}} = \mathbf{g}\mathbf{out} \cdot \frac{\partial f(\mathbf{x}\mathbf{in}, \mathbf{W})}{\partial \mathbf{W}} = \mathbf{g}\mathbf{out} \cdot \frac{\partial \mathbf{x}\mathbf{out}}{\partial \mathbf{W}}$$

- g_{out}은 현재 layer의 출력 f에 대한 손실 J의 gradient임.
- 뒤쪽 layer에서 전달된 이 gradient에 현재 layer의 local derivative를 결합하는 것이 chain rule의 핵심임.
- 이를 통해 입력에 대한 gradient $g_{in} = \partial J / \partial x$뿐 아니라 weight W와 bias b에 대한 gradient도 계산함.
- 즉, g_{out}은 chain rule 자체가 아니라, chain rule을 적용할 때 출력 쪽에서 전달받는 gradient임.

$$\frac{\partial J}{\partial \mathbf{W}_{ij}} = \frac{\partial J}{\partial \mathbf{x}\mathbf{out}_i} \cdot \frac{\partial \mathbf{x}\mathbf{out}_i}{\partial \mathbf{W}_{ij}} = \frac{\partial J}{\partial \mathbf{x}\mathbf{out}_i} \cdot \mathbf{x}\mathbf{in}_j$$

$$\frac{\partial \mathbf{x}\mathbf{out}_i}{\partial \mathbf{W}_{ij}} = \mathbf{x}\mathbf{in}_j$$

$$\frac{\partial J}{\partial \mathbf{W}} = \mathbf{x}_{\mathbf{in}} \cdot \frac{\partial J}{\partial \mathbf{x}_{\mathbf{out}}} = \mathbf{x}_{\mathbf{in}} \cdot \mathbf{g}_{\mathbf{out}}$$

- ※ 정정: 캡처 15의 성분식 $\partial J/\partial W_{ij} = (g_{out})_i (x_{in})_j$가 기준임. 행렬로는 $\partial J/\partial W = g_{out}\, x_{in}^{\top}$ (외적)이 맞음. 캡처 16의 $x_{in} \cdot g_{out}$는 일반적인 행렬곱 순서로 해석하면 안 됨.
- update 할 때 위 식을 쓰게 됨.

Differentiable programming

- 미분 가능한 연산들을 조합해 프로그램을 구성하면, automatic differentiation을 통해 gradient를 계산할 수 있음.
- 따라서 gradient-based optimization을 적용하기 쉬워질 수 있음.
- 하지만 미분 가능하다는 것이 최적화 자체가 항상 쉽다는 뜻은 아님.
- 실제 최적화 난이도는 손실 함수의 형태, 초기화, learning rate 등에 따라 달라짐.
- 특히 비볼록(non-convex) 손실 함수에서는 local minimum이나 saddle point 등의 문제가 존재할 수 있으며, 전역 최소값에 도달하는 것은 보장되지 않음.
- compositional “block based programming”